

BraTS 2025 GLI — 3D U-Net Tumor Segmentation

Author: Ganji Revanth Kumar

Data: BraTS 2025 GLI pre-operative challenge training set, 1251 cases

Environment: Kaggle, single GPU, TensorFlow / Keras 3

What this notebook does

Multi-modal brain MRI (four sequences per patient) goes in, a voxel-wise tumor label map comes out. The pipeline is:

```
NIfTI volumes (182x218x182, 4 modalities)
-> per-modality z-score over brain voxels only
-> resample to 48x48x48
-> cache as .npy
-> tf.data pipeline
-> 3D U-Net (3 levels, skip connections, soft Dice loss)
-> voxel-wise class probabilities
```

Why this sits on an engineering CV

The physics is unrelated, but the *object* is not: this is 3D deep learning on a gridded volumetric field with multiple input channels. Swap MRI intensities for a voxelised stress or velocity field and the machinery is the same. That was the reason I built it.

A note on the reported metrics

Sections 9 and 10 matter most if you are reviewing this critically. The Dice and IoU numbers produced in Section 9 are **inflated**, and Section 10 explains why and gives the corrected evaluation. I found the problem after the fact. The original code and its outputs are left in place rather than quietly replaced, because the failure mode is a common and instructive one.

All outputs below are from the original run. Nothing has been re-executed.

1. Dataset layout

Each case is a directory containing five NIfTI files: four MRI modalities and one expert-annotated segmentation mask.

- `t1n` — T1-weighted, native
- `t1c` — T1-weighted, post-contrast (enhancing tumor shows up here)
- `t2w` — T2-weighted
- `t2f` — T2-FLAIR (edema shows up here)
- `seg` — ground truth label map

Different tumor tissues are visible in different sequences, which is why all four are fed to the network as channels rather than picking one.

```
import os

data_root = '/kaggle/input/bratstask-1/BraTS2025-GLI-PRE-Challenge-TrainingData/'
case_dirs = sorted([
    os.path.join(data_root, d)
    for d in os.listdir(data_root)
    if os.path.isdir(os.path.join(data_root, d))
])
print(f"Found {len(case_dirs)} cases")
print(f"Example case folder: {case_dirs[0]}")
print("Files in example case folder:", os.listdir(case_dirs[0]))
```

```
Found 1251 cases
Example case folder: /kaggle/input/bratstask-1/BraTS2025-GLI-PRE-Challenge-TrainingData/BraTS-GLI-00000-000
Files in example case folder: ['BraTS-GLI-00000-000-t1n.nii', 'BraTS-GLI-00000-000-t2w.nii', 'BraTS-GLI-00000-000-t1c.nii', 'BraTS-GLI-00000-000-t2f.nii', 'BraTS-GLI-00000-000-seg.nii']
```

Loading a single case

Stacking the four modalities on the last axis gives a (H, W, D, 4) array — the direct 3D analogue of an RGB image, with four channels instead of three.

Note the printed label values: this case contains only [0, 2, 3]. Label 1 is absent. BraTS label sparsity varies case to case, which becomes important in Section 10.

```
import nibabel as nib
import numpy as np
import os

# Correct folder path
sample_case = '/kaggle/input/bratstask-1/BraTS2025-GLI-PRE-Challenge-TrainingData/BraTS-GLI-00540-000'

# Modalities to load
modalities = ['t1n', 't1c', 't2w', 't2f']
images = []
for mod in modalities:
    file_name = [f for f in os.listdir(sample_case) if f.endswith(f'-{mod}.nii')][0]
    img = nib.load(os.path.join(sample_case, file_name)).get_fdata()
    images.append(img)
image = np.stack(images, axis=-1) # shape: (H, W, D, 4)

# Load segmentation mask
mask_file = [f for f in os.listdir(sample_case) if f.endswith('-seg.nii')][0]
mask = nib.load(os.path.join(sample_case, mask_file)).get_fdata() # shape: (H, W, D)

print(f"image shape: {image.shape} mask shape: {mask.shape} mask labels: {np.unique(mask)}")
```

```
image shape: (182, 218, 182, 4) mask shape: (182, 218, 182) mask labels: [0. 2. 3.]
```

Visual check

Never trust a loader without looking at what it produced. Same central slice across all four modalities plus the mask. The contrast differences between sequences are exactly what the network exploits.

```

import nibabel as nib
import numpy as np
import matplotlib.pyplot as plt
import os

# Path to your data, example case directory
data_root = '/kaggle/input/bratstask-1/BraTS2025-GLI-PRE-Challenge-TrainingData/'
example_case_dir = os.path.join(data_root, 'BraTS-GLI-00000-000')

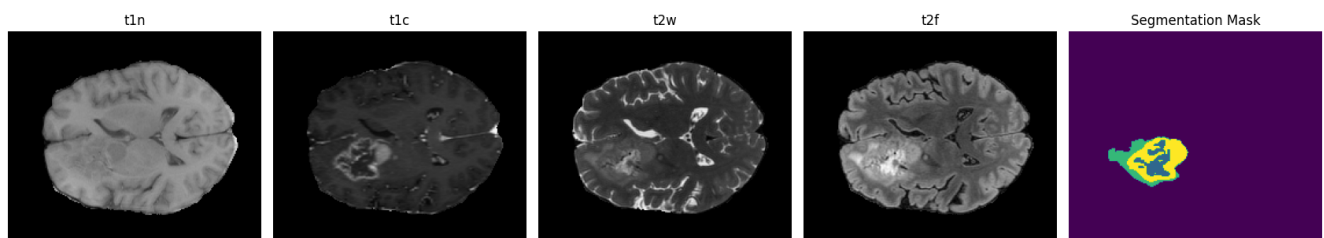
# List the files (already validated)
files = os.listdir(example_case_dir)

# Helper to auto-grab filenames regardless of their specific numeric ID
def pick(suffix): return [f for f in files if f.endswith(suffix)][0]

# Load all four modalities and segmentation mask
modalities = ['t1n', 't1c', 't2w', 't2f']
img_data = []
for mod in modalities:
    fname = pick(f'-{mod}.nii')
    img = nib.load(os.path.join(example_case_dir, fname)).get_fdata()
    img_data.append(img)
mask = nib.load(os.path.join(example_case_dir, pick('-seg.nii'))).get_fdata()

# Show a central slice from each input and the mask
slice_idx = mask.shape[2] // 2
plt.figure(figsize=(17, 4))
for i, (mod, img) in enumerate(zip(modalities, img_data)):
    plt.subplot(1, 5, i+1)
    plt.imshow(img[:, :, slice_idx], cmap='gray')
    plt.title(mod)
    plt.axis('off')
plt.subplot(1, 5, 5)
plt.imshow(mask[:, :, slice_idx], cmap='viridis')
plt.title('Segmentation Mask')
plt.axis('off')
plt.tight_layout()
plt.show()

```



2. Label handling

The intent here was to compact the label values so they could be one-hot encoded into three channels.

This cell contains a bug, left in place for honesty. Original label 1 is never touched, and original label 2 is also mapped to 1. So two distinct tissue classes end up merged into a single output channel. The model was trained on three merged classes rather than the four the dataset provides.

The printed output `[0, 1, 2]` looks correct only because the sample case happened not to contain label 1.

```
def remap_labels(mask):
    # Remap mask for training: 0 → 0 (background), 2 → 1, 3 → 2 (shift down)
    mask_remapped = mask.copy()
    mask_remapped[mask == 2] = 1
    mask_remapped[mask == 3] = 2
    return mask_remapped

remapped_mask = remap_labels(mask)
print('Remapped mask classes:', np.unique(remapped_mask))
```

```
Remapped mask classes: [0. 1. 2.]
```

3. Preprocessing and caching

Two decisions worth explaining:

Z-score per modality, over brain voxels only. MRI intensities have no absolute units — the same tissue gives different numbers on different scanners, so each volume has to be standardised against itself. The brain mask (> 0) matters because roughly 80% of each volume is air. Including it would drag the mean toward zero and shrink the standard deviation, compressing all the tissue contrast the network needs.

Nearest-neighbour resampling for the mask (`order=0`), trilinear for the image (`order=1`). Interpolating a label map would invent fractional label values that correspond to no tissue at all.

Downsampling to 48^3 is the biggest compromise in this notebook. It is roughly a 4x reduction per axis from the native 182^3 , done to fit training in a Kaggle session. Small structures — thin enhancing rims especially — get partially averaged away. Any metric reported here is therefore not comparable to BraTS leaderboard numbers, which are computed at full resolution.

Preprocessing runs once and writes `.npy` files, so training epochs are not paying the NIfTI decode and resample cost repeatedly.

```
import os
import numpy as np
import nibabel as nib
from scipy.ndimage import zoom
import tensorflow as tf
from sklearn.model_selection import train_test_split
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
from tensorflow.keras import layers, Model
```

```
2025-08-07 14:13:52.392094: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1754576032.580853      36 cuda_dnn.cc:8310] Unable to register cuDNN factory: Attempting to
register factory for plugin cuDNN when one has already been registered
E0000 00:00:1754576032.637263      36 cuda_blas.cc:1418] Unable to register cuBLAS factory: Attempting to
register factory for plugin cuBLAS when one has already been registered
```

```
from tensorflow.keras import mixed_precision
mixed_precision.set_global_policy('mixed_float16')
```

```

data_root = '/kaggle/input/bratstask-1/BraTS2025-GLI-PRE-Challenge-TrainingData/'
target_shape = (48, 48, 48)

def preprocess_and_save_case(case_dir, target_shape, save_dir):
    os.makedirs(save_dir, exist_ok=True)
    modalities = ['t1n', 't1c', 't2w', 't2f']
    images = []
    files = os.listdir(case_dir)
    for mod in modalities:
        file_name = [f for f in files if f.endswith(f'-{mod}.nii')][0]
        img = nib.load(os.path.join(case_dir, file_name)).get_fdata()
        images.append(img)
    image = np.stack(images, axis=-1)
    mask_file = [f for f in files if f.endswith('-seg.nii')][0]
    mask = nib.load(os.path.join(case_dir, mask_file)).get_fdata()
    mask_remapped = mask.copy()
    mask_remapped[mask == 2] = 1
    mask_remapped[mask == 3] = 2

    # Z-score normalization (per modality, only brain)
    image_norm = image.copy()
    for i in range(image.shape[-1]):
        modality = image[..., i]
        brain_mask = modality > 0
        mean = modality[brain_mask].mean() if np.any(brain_mask) else 0
        std = modality[brain_mask].std() if np.any(brain_mask) else 1
        image_norm[..., i][brain_mask] = (modality[brain_mask] - mean) / (std + 1e-8)

    # Resize to target shape
    factors = [t / s for t, s in zip(target_shape, image_norm.shape[:3])]
    resized_image = np.zeros(target_shape + (image_norm.shape[3],), dtype=np.float32)
    for i in range(image_norm.shape[3]):
        resized_image[..., i] = zoom(image_norm[..., i], factors, order=1)
    resized_mask = zoom(mask_remapped, factors, order=0).astype(np.uint8)

    np.save(os.path.join(save_dir, 'input.npy'), resized_image)
    np.save(os.path.join(save_dir, 'label.npy'), resized_mask)
    return os.path.join(save_dir, 'input.npy'), os.path.join(save_dir, 'label.npy')

# List directories and split
case_dirs = sorted([os.path.join(data_root, d) for d in os.listdir(data_root) if
os.path.isdir(os.path.join(data_root, d))])
train_dirs, val_dirs = train_test_split(case_dirs, test_size=0.1, random_state=42)

train_save_root = '/kaggle/working/train'
val_save_root = '/kaggle/working/val'
train_inputs, train_labels, val_inputs, val_labels = [], [], [], []

for case_dir in train_dirs:
    case_name = os.path.basename(case_dir)
    save_dir = os.path.join(train_save_root, case_name)
    input_path, label_path = preprocess_and_save_case(case_dir, target_shape, save_dir)
    train_inputs.append(input_path)
    train_labels.append(label_path)

for case_dir in val_dirs:
    case_name = os.path.basename(case_dir)
    save_dir = os.path.join(val_save_root, case_name)
    input_path, label_path = preprocess_and_save_case(case_dir, target_shape, save_dir)
    val_inputs.append(input_path)
    val_labels.append(label_path)

print("Done preprocessing and saving .npy files.")

```

Done preprocessing and saving .npy files.

4. Input pipeline

`tf.data` reads the cached arrays, one-hot encodes the labels, batches, and prefetches so that data loading overlaps with GPU compute.

`tf.py_function` is used because `np.load` is not a TensorFlow op. This is the pragmatic choice, not the fast one — it holds the Python GIL, so it caps parallelism. A TFRecord pipeline would be faster and is the obvious next step for a larger run.

Batch size is 2, which is what a $48^3 \times 4$ volume plus a 3-channel output allows on a single Kaggle GPU.

```
import tensorflow as tf

def load_npy_data(input_path, label_path):
    X = np.load(input_path.numpy().decode('utf-8')).astype(np.float32)
    y = np.load(label_path.numpy().decode('utf-8')).astype(np.uint8)
    y_cat = tf.keras.utils.to_categorical(y, num_classes=3)
    return X, y_cat

def tf_load_npy_data(input_path, label_path):
    X, y = tf.py_function(load_npy_data, [input_path, label_path], [tf.float32, tf.float32])
    X.set_shape([48, 48, 48, 4])
    y.set_shape([48, 48, 48, 3])
    return X, y

batch_size = 2

train_dataset = tf.data.Dataset.from_tensor_slices((train_inputs, train_labels))
train_dataset = train_dataset.shuffle(len(train_inputs))
train_dataset = train_dataset.map(tf_load_npy_data, num_parallel_calls=tf.data.AUTOTUNE)
train_dataset = train_dataset.batch(batch_size)
train_dataset = train_dataset.prefetch(tf.data.AUTOTUNE)

val_dataset = tf.data.Dataset.from_tensor_slices((val_inputs, val_labels))
val_dataset = val_dataset.map(tf_load_npy_data, num_parallel_calls=tf.data.AUTOTUNE)
val_dataset = val_dataset.batch(batch_size)
val_dataset = val_dataset.prefetch(tf.data.AUTOTUNE)

print("tf.data.Dataset ready.")
```

```
tf.data.Dataset ready.
```

5. Model and loss

Architecture

A 3-level 3D U-Net. The encoder halves spatial resolution three times while doubling channel width ($32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ at the bottleneck), and the decoder mirrors it with transposed convolutions.

Skip connections are the whole point of a U-Net. Pooling gives the network enough receptive field to know *what* it is looking at, but destroys the precise voxel boundaries. The skip connections carry high-resolution spatial detail from each encoder level directly across to the matching decoder level, so the output can be both semantically correct and spatially sharp. Without them the segmentation comes out as blurred blobs.

`he_normal` initialisation pairs with ReLU. Batch norm after each convolution keeps activations well-scaled through a fairly deep stack. Dropout 0.3 at each

level, which is aggressive but the training set is small relative to the parameter count.

Soft Dice loss

Class imbalance here is extreme: tumour is a small fraction of the volume, and enhancing tumour smaller still. Cross-entropy on this data lets the network reach very high accuracy by predicting "background" almost everywhere. The training log in Section 7 shows exactly that — 99.7% accuracy, which is meaningless.

Dice is a set-overlap measure, so it is insensitive to how much background there is. Making it differentiable means using the predicted probabilities directly instead of thresholding, hence "soft". The `smooth` term prevents division by zero when a class is absent from a batch — and in Section 10 that same term turns out to be what inflated the reported score.

```
def conv3d_block(input_tensor, n_filters, kernel_size=3, batchnorm=True):
    x = layers.Conv3D(filters=n_filters, kernel_size=kernel_size,
                      kernel_initializer='he_normal', padding='same')(input_tensor)
    if batchnorm:
        x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    x = layers.Conv3D(filters=n_filters, kernel_size=kernel_size,
                      kernel_initializer='he_normal', padding='same')(x)
    if batchnorm:
        x = layers.BatchNormalization()(x)
    x = layers.Activation('relu')(x)
    return x

def build_3d_unet(input_shape=(48,48,48,4), n_filters=32, n_classes=3, dropout=0.3):
    inputs = layers.Input(input_shape)
    c1 = conv3d_block(inputs, n_filters)
    p1 = layers.MaxPooling3D((2, 2, 2))(c1)
    p1 = layers.Dropout(dropout)(p1)
    c2 = conv3d_block(p1, n_filters*2)
    p2 = layers.MaxPooling3D((2, 2, 2))(c2)
    p2 = layers.Dropout(dropout)(p2)
    c3 = conv3d_block(p2, n_filters*4)
    p3 = layers.MaxPooling3D((2, 2, 2))(c3)
    p3 = layers.Dropout(dropout)(p3)
    c4 = conv3d_block(p3, n_filters*8)
    u5 = layers.Conv3DTranspose(n_filters*4, (2,2,2), strides=(2,2,2), padding='same')(c4)
    u5 = layers.concatenate([u5, c3])
    u5 = layers.Dropout(dropout)(u5)
    c5 = conv3d_block(u5, n_filters*4)
    u6 = layers.Conv3DTranspose(n_filters*2, (2,2,2), strides=(2,2,2), padding='same')(c5)
    u6 = layers.concatenate([u6, c2])
    u6 = layers.Dropout(dropout)(u6)
    c6 = conv3d_block(u6, n_filters*2)
    u7 = layers.Conv3DTranspose(n_filters, (2,2,2), strides=(2,2,2), padding='same')(c6)
    u7 = layers.concatenate([u7, c1])
    u7 = layers.Dropout(dropout)(u7)
    c7 = conv3d_block(u7, n_filters)
    outputs = layers.Conv3D(n_classes, (1,1,1), activation='softmax')(c7)
    model = Model(inputs=[inputs], outputs=[outputs])
    return model

def dice_loss(y_true, y_pred, smooth=1e-6):
    axes = tuple(range(1, len(y_true.shape)-1))
    intersection = tf.reduce_sum(y_true * y_pred, axis=axes)
    union = tf.reduce_sum(y_true + y_pred, axis=axes)
    dice = (2. * intersection + smooth) / (union + smooth)
    return 1 - tf.reduce_mean(dice)
```

Callbacks

`ModelCheckpoint` on `val_loss` keeps the best model rather than the last.

`EarlyStopping` with `restore_best_weights` guards against late overfitting.

`ReduceLROnPlateau` halves the learning rate when validation stalls.

The custom periodic-save callback exists for a practical reason: Kaggle sessions have a wall-clock limit, so training had to survive being interrupted and resumed.

```
from tensorflow.keras.callbacks import Callback, ModelCheckpoint, EarlyStopping, ReduceLROnPlateau

# Define callbacks list first
callbacks = [
    ModelCheckpoint('best_model.h5', save_best_only=True, monitor='val_loss'),
    EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5)
]

# Define and add your custom callback
class SaveWeightsEveryNEpoch(Callback):
    def __init__(self, save_freq=5):
        super().__init__()
        self.save_freq = save_freq
    def on_epoch_end(self, epoch, logs=None):
        if (epoch + 1) % self.save_freq == 0:
            filename = f'model_weights_epoch_{epoch+1:03d}.weights.h5'
            self.model.save_weights(filename)
            print(f"Saved weights to {filename}")

save_weights_callback = SaveWeightsEveryNEpoch(save_freq=5)
callbacks.append(save_weights_callback)

# Model compilation and training as before...
```

6. Training

Model summary and the first training block. 563 steps per epoch at batch size 2, so ~1126 training volumes.

Read the accuracy figure with suspicion. 0.9975 is what you get from labelling almost everything background. The loss value is the informative one.


```

input_shape = (48,48,48,4)
model = build_3d_unet(input_shape, n_filters=32, n_classes=3, dropout=0.3)
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
              loss=dice_loss,
              metrics=['accuracy'])
model.summary()

history = model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=100,
    callbacks=callbacks,
    verbose=1
)

```

Model: "functional_1"

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 48, 48, 4)	0	-
conv3d_15 (Conv3D)	(None, 48, 48, 48, 32)	3,488	input_layer_1[0]...
batch_normalizatio... (BatchNormalizatio...)	(None, 48, 48, 48, 32)	128	conv3d_15[0][0]
activation_14 (Activation)	(None, 48, 48, 48, 32)	0	batch_normalizat...
conv3d_16 (Conv3D)	(None, 48, 48, 48, 32)	27,680	activation_14[0]...
batch_normalizatio... (BatchNormalizatio...)	(None, 48, 48, 48, 32)	128	conv3d_16[0][0]
activation_15 (Activation)	(None, 48, 48, 48, 32)	0	batch_normalizat...
max_pooling3d_3 (MaxPooling3D)	(None, 24, 24, 24, 32)	0	activation_15[0]...
dropout_6 (Dropout)	(None, 24, 24, 24, 32)	0	max_pooling3d_3[...
conv3d_17 (Conv3D)	(None, 24, 24, 24, 64)	55,360	dropout_6[0][0]
batch_normalizatio... (BatchNormalizatio...)	(None, 24, 24, 24, 64)	256	conv3d_17[0][0]
activation_16 (Activation)	(None, 24, 24, 24, 64)	0	batch_normalizat...
conv3d_18 (Conv3D)	(None, 24, 24, 24, 64)	110,656	activation_16[0]...
batch_normalizatio... (BatchNormalizatio...)	(None, 24, 24, 24, 64)	256	conv3d_18[0][0]
activation_17 (Activation)	(None, 24, 24, 24, 64)	0	batch_normalizat...
max_pooling3d_4 (MaxPooling3D)	(None, 12, 12, 12, 64)	0	activation_17[0]...
dropout_7 (Dropout)	(None, 12, 12, 12, 64)	0	max_pooling3d_4[...
conv3d_19 (Conv3D)	(None, 12, 12, 12, 128)	221,312	dropout_7[0][0]
batch_normalizatio... (BatchNormalizatio...)	(None, 12, 12, 12, 128)	512	conv3d_19[0][0]
activation_18 (Activation)	(None, 12, 12, 12, 128)	0	batch_normalizat...
conv3d_20 (Conv3D)	(None, 12, 12, 12, 128)	442,496	activation_18[0]...
batch_normalizatio... (BatchNormalizatio...)	(None, 12, 12, 12, 128)	512	conv3d_20[0][0]

activation_19 (Activation)	(None, 12, 12, 12, 128)	0	batch_normalizat...
max_pooling3d_5 (MaxPooling3D)	(None, 6, 6, 6, 128)	0	activation_19[0]...
dropout_8 (Dropout)	(None, 6, 6, 6, 128)	0	max_pooling3d_5[...
conv3d_21 (Conv3D)	(None, 6, 6, 6, 256)	884,992	dropout_8[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 6, 6, 6, 256)	1,024	conv3d_21[0][0]
activation_20 (Activation)	(None, 6, 6, 6, 256)	0	batch_normalizat...
conv3d_22 (Conv3D)	(None, 6, 6, 6, 256)	1,769,728	activation_20[0]...
batch_normalizatio... (BatchNormalizatio...	(None, 6, 6, 6, 256)	1,024	conv3d_22[0][0]
activation_21 (Activation)	(None, 6, 6, 6, 256)	0	batch_normalizat...
conv3d_transpose_3 (Conv3DTranspose)	(None, 12, 12, 12, 128)	262,272	activation_21[0]...
concatenate_3 (Concatenate)	(None, 12, 12, 12, 256)	0	conv3d_transpose... activation_19[0]...
dropout_9 (Dropout)	(None, 12, 12, 12, 256)	0	concatenate_3[0]...
conv3d_23 (Conv3D)	(None, 12, 12, 12, 128)	884,864	dropout_9[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 12, 12, 12, 128)	512	conv3d_23[0][0]
activation_22 (Activation)	(None, 12, 12, 12, 128)	0	batch_normalizat...
conv3d_24 (Conv3D)	(None, 12, 12, 12, 128)	442,496	activation_22[0]...
batch_normalizatio... (BatchNormalizatio...	(None, 12, 12, 12, 128)	512	conv3d_24[0][0]
activation_23 (Activation)	(None, 12, 12, 12, 128)	0	batch_normalizat...
conv3d_transpose_4 (Conv3DTranspose)	(None, 24, 24, 24, 64)	65,600	activation_23[0]...
concatenate_4 (Concatenate)	(None, 24, 24, 24, 128)	0	conv3d_transpose... activation_17[0]...
dropout_10 (Dropout)	(None, 24, 24, 24, 128)	0	concatenate_4[0]...
conv3d_25 (Conv3D)	(None, 24, 24, 24, 64)	221,248	dropout_10[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 24, 24, 24, 64)	256	conv3d_25[0][0]
activation_24 (Activation)	(None, 24, 24, 24, 64)	0	batch_normalizat...
conv3d_26 (Conv3D)	(None, 24, 24, 24, 64)	110,656	activation_24[0]...
batch_normalizatio... (BatchNormalizatio...	(None, 24, 24, 24, 64)	256	conv3d_26[0][0]
activation_25 (Activation)	(None, 24, 24, 24, 64)	0	batch_normalizat...
conv3d_transpose_5 (Conv3DTranspose)	(None, 48, 48, 48, 32)	16,416	activation_25[0]...
concatenate_5 (Concatenate)	(None, 48, 48, 48, 64)	0	conv3d_transpose... activation_15[0]...
dropout_11 (Dropout)	(None, 48, 48, 48, 64)	0	concatenate_5[0]...
conv3d_27 (Conv3D)	(None, 48, 48, 48, 64)	55,328	dropout_11[0][0]

	48, 32)		
batch_normalizatio... (BatchNormalizatio...	(None, 48, 48, 48, 32)	128	conv3d_27[0][0]
activation_26 (Activation)	(None, 48, 48, 48, 32)	0	batch_normalizat...
conv3d_28 (Conv3D)	(None, 48, 48, 48, 32)	27,680	activation_26[0]...
batch_normalizatio... (BatchNormalizatio...	(None, 48, 48, 48, 32)	128	conv3d_28[0][0]
activation_27 (Activation)	(None, 48, 48, 48, 32)	0	batch_normalizat...
conv3d_29 (Conv3D)	(None, 48, 48, 48, 3)	99	activation_27[0]...

Total params: 5,608,003 (21.39 MB)

Trainable params: 5,605,187 (21.38 MB)

Non-trainable params: 2,816 (11.00 KB)

Epoch 1/100

```
/usr/local/lib/python3.11/dist-packages/keras/src/models/functional.py:237: UserWarning: The structure of
`inputs` doesn't match the expected structure.
Expected: ['keras_tensor_59']
Received: inputs=Tensor(shape=(None, 48, 48, 48, 4))
warnings.warn(msg)
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1754580338.834194    104 service.cc:148] XLA service 0x78c394039380 initialized for platform CUDA
(this does not guarantee that XLA will be used). Devices:
I0000 00:00:1754580338.834884    104 service.cc:156]   StreamExecutor device (0): Tesla T4, Compute Capability
7.5
I0000 00:00:1754580338.834925    104 service.cc:156]   StreamExecutor device (1): Tesla T4, Compute Capability
7.5
I0000 00:00:1754580340.291548    104 cuda_dnn.cc:529] Loaded cuDNN version 90300
I0000 00:00:1754580357.036863    104 device_compiler.h:188] Compiled cluster using XLA! This line is logged
at most once for the lifetime of the process.
```

```
563/563 ===== 143s 197ms/step - accuracy: 0.9222 - loss: 0.6416 - val_accuracy: 0.9903 -
val_loss: 0.4943 - learning_rate: 1.0000e-04
Epoch 2/100
563/563 ===== 97s 172ms/step - accuracy: 0.9907 - loss: 0.4603 - val_accuracy: 0.9955 -
val_loss: 0.2981 - learning_rate: 1.0000e-04
Epoch 3/100
563/563 ===== 96s 171ms/step - accuracy: 0.9952 - loss: 0.2686 - val_accuracy: 0.9960 -
val_loss: 0.2146 - learning_rate: 1.0000e-04
Epoch 4/100
563/563 ===== 96s 171ms/step - accuracy: 0.9960 - loss: 0.2006 - val_accuracy: 0.9959 -
val_loss: 0.2000 - learning_rate: 1.0000e-04
Epoch 5/100
563/563 ===== 0s 163ms/step - accuracy: 0.9962 - loss: 0.1809Saved weights to
model_weights_epoch_005.weights.h5
563/563 ===== 96s 171ms/step - accuracy: 0.9962 - loss: 0.1809 - val_accuracy: 0.9964 -
val_loss: 0.1778 - learning_rate: 1.0000e-04
Epoch 6/100
563/563 ===== 97s 172ms/step - accuracy: 0.9965 - loss: 0.1692 - val_accuracy: 0.9966 -
val_loss: 0.1624 - learning_rate: 1.0000e-04
Epoch 7/100
563/563 ===== 96s 170ms/step - accuracy: 0.9966 - loss: 0.1623 - val_accuracy: 0.9966 -
val_loss: 0.1692 - learning_rate: 1.0000e-04
Epoch 8/100
563/563 ===== 97s 171ms/step - accuracy: 0.9966 - loss: 0.1598 - val_accuracy: 0.9968 -
val_loss: 0.1555 - learning_rate: 1.0000e-04
Epoch 9/100
563/563 ===== 96s 171ms/step - accuracy: 0.9968 - loss: 0.1492 - val_accuracy: 0.9968 -
val_loss: 0.1613 - learning_rate: 1.0000e-04
Epoch 10/100
563/563 ===== 0s 164ms/step - accuracy: 0.9967 - loss: 0.1458Saved weights to
model_weights_epoch_010.weights.h5
563/563 ===== 97s 172ms/step - accuracy: 0.9967 - loss: 0.1458 - val_accuracy: 0.9967 -
val_loss: 0.1536 - learning_rate: 1.0000e-04
Epoch 11/100
563/563 ===== 96s 171ms/step - accuracy: 0.9969 - loss: 0.1410 - val_accuracy: 0.9968 -
val_loss: 0.1599 - learning_rate: 1.0000e-04
Epoch 12/100
```

```

563/563 ===== 97s 172ms/step - accuracy: 0.9969 - loss: 0.1493 - val_accuracy: 0.9968 -
val_loss: 0.1511 - learning_rate: 1.0000e-04
Epoch 13/100
563/563 ===== 97s 172ms/step - accuracy: 0.9968 - loss: 0.1432 - val_accuracy: 0.9969 -
val_loss: 0.1433 - learning_rate: 1.0000e-04
Epoch 14/100
563/563 ===== 97s 172ms/step - accuracy: 0.9970 - loss: 0.1341 - val_accuracy: 0.9969 -
val_loss: 0.1509 - learning_rate: 1.0000e-04
Epoch 15/100
563/563 ===== 0s 164ms/step - accuracy: 0.9970 - loss: 0.1337Saved weights to
model_weights_epoch_015.weights.h5
563/563 ===== 97s 172ms/step - accuracy: 0.9970 - loss: 0.1337 - val_accuracy: 0.9969 -
val_loss: 0.1430 - learning_rate: 1.0000e-04
Epoch 16/100
563/563 ===== 96s 171ms/step - accuracy: 0.9971 - loss: 0.1322 - val_accuracy: 0.9971 -
val_loss: 0.1471 - learning_rate: 1.0000e-04
Epoch 17/100
563/563 ===== 97s 172ms/step - accuracy: 0.9972 - loss: 0.1301 - val_accuracy: 0.9971 -
val_loss: 0.1403 - learning_rate: 1.0000e-04
Epoch 18/100
563/563 ===== 97s 171ms/step - accuracy: 0.9970 - loss: 0.1318 - val_accuracy: 0.9969 -
val_loss: 0.1405 - learning_rate: 1.0000e-04
Epoch 19/100
563/563 ===== 96s 171ms/step - accuracy: 0.9971 - loss: 0.1324 - val_accuracy: 0.9971 -
val_loss: 0.1410 - learning_rate: 1.0000e-04
Epoch 20/100
563/563 ===== 0s 164ms/step - accuracy: 0.9971 - loss: 0.1289Saved weights to
model_weights_epoch_020.weights.h5
563/563 ===== 97s 172ms/step - accuracy: 0.9971 - loss: 0.1289 - val_accuracy: 0.9970 -
val_loss: 0.1460 - learning_rate: 1.0000e-04
Epoch 21/100
563/563 ===== 96s 171ms/step - accuracy: 0.9972 - loss: 0.1258 - val_accuracy: 0.9970 -
val_loss: 0.1482 - learning_rate: 1.0000e-04
Epoch 22/100
563/563 ===== 97s 172ms/step - accuracy: 0.9972 - loss: 0.1207 - val_accuracy: 0.9972 -
val_loss: 0.1356 - learning_rate: 1.0000e-04
Epoch 23/100
563/563 ===== 96s 171ms/step - accuracy: 0.9973 - loss: 0.1217 - val_accuracy: 0.9972 -
val_loss: 0.1378 - learning_rate: 1.0000e-04
Epoch 24/100
563/563 ===== 96s 171ms/step - accuracy: 0.9972 - loss: 0.1213 - val_accuracy: 0.9972 -
val_loss: 0.1439 - learning_rate: 1.0000e-04
Epoch 25/100
563/563 ===== 0s 164ms/step - accuracy: 0.9972 - loss: 0.1225Saved weights to
model_weights_epoch_025.weights.h5
563/563 ===== 97s 172ms/step - accuracy: 0.9972 - loss: 0.1225 - val_accuracy: 0.9970 -
val_loss: 0.1403 - learning_rate: 1.0000e-04
Epoch 26/100
563/563 ===== 96s 171ms/step - accuracy: 0.9974 - loss: 0.1187 - val_accuracy: 0.9972 -
val_loss: 0.1359 - learning_rate: 1.0000e-04
Epoch 27/100
563/563 ===== 96s 171ms/step - accuracy: 0.9973 - loss: 0.1201 - val_accuracy: 0.9971 -
val_loss: 0.1458 - learning_rate: 1.0000e-04
Epoch 28/100
563/563 ===== 96s 171ms/step - accuracy: 0.9972 - loss: 0.1175 - val_accuracy: 0.9973 -
val_loss: 0.1367 - learning_rate: 5.0000e-05
Epoch 29/100
563/563 ===== 96s 171ms/step - accuracy: 0.9974 - loss: 0.1154 - val_accuracy: 0.9972 -
val_loss: 0.1364 - learning_rate: 5.0000e-05
Epoch 30/100
563/563 ===== 0s 164ms/step - accuracy: 0.9974 - loss: 0.1137Saved weights to
model_weights_epoch_030.weights.h5
563/563 ===== 97s 172ms/step - accuracy: 0.9974 - loss: 0.1137 - val_accuracy: 0.9973 -
val_loss: 0.1336 - learning_rate: 5.0000e-05
Epoch 31/100
563/563 ===== 97s 171ms/step - accuracy: 0.9974 - loss: 0.1128 - val_accuracy: 0.9973 -
val_loss: 0.1343 - learning_rate: 5.0000e-05
Epoch 32/100
563/563 ===== 97s 172ms/step - accuracy: 0.9974 - loss: 0.1112 - val_accuracy: 0.9973 -
val_loss: 0.1321 - learning_rate: 5.0000e-05
Epoch 33/100
563/563 ===== 96s 171ms/step - accuracy: 0.9974 - loss: 0.1102 - val_accuracy: 0.9973 -
val_loss: 0.1325 - learning_rate: 5.0000e-05
Epoch 34/100
563/563 ===== 96s 171ms/step - accuracy: 0.9974 - loss: 0.1152 - val_accuracy: 0.9973 -
val_loss: 0.1333 - learning_rate: 5.0000e-05
Epoch 35/100
563/563 ===== 0s 164ms/step - accuracy: 0.9975 - loss: 0.1077Saved weights to
model_weights_epoch_035.weights.h5

```

```

563/563 ===== 97s 172ms/step - accuracy: 0.9975 - loss: 0.1077 - val_accuracy: 0.9973 -
val_loss: 0.1311 - learning_rate: 5.0000e-05
Epoch 36/100
563/563 ===== 96s 171ms/step - accuracy: 0.9976 - loss: 0.1113 - val_accuracy: 0.9972 -
val_loss: 0.1338 - learning_rate: 5.0000e-05
Epoch 37/100
563/563 ===== 96s 171ms/step - accuracy: 0.9975 - loss: 0.1134 - val_accuracy: 0.9974 -
val_loss: 0.1321 - learning_rate: 5.0000e-05
Epoch 38/100
563/563 ===== 97s 171ms/step - accuracy: 0.9975 - loss: 0.1113 - val_accuracy: 0.9973 -
val_loss: 0.1357 - learning_rate: 5.0000e-05
Epoch 39/100
563/563 ===== 97s 172ms/step - accuracy: 0.9974 - loss: 0.1140 - val_accuracy: 0.9973 -
val_loss: 0.1310 - learning_rate: 5.0000e-05
Epoch 40/100
563/563 ===== 0s 164ms/step - accuracy: 0.9975 - loss: 0.1114Saved weights to
model_weights_epoch_040.weights.h5
563/563 ===== 97s 172ms/step - accuracy: 0.9975 - loss: 0.1114 - val_accuracy: 0.9974 -
val_loss: 0.1309 - learning_rate: 5.0000e-05
Epoch 41/100
563/563 ===== 96s 171ms/step - accuracy: 0.9975 - loss: 0.1066 - val_accuracy: 0.9973 -
val_loss: 0.1340 - learning_rate: 5.0000e-05
Epoch 42/100
563/563 ===== 97s 171ms/step - accuracy: 0.9975 - loss: 0.1068 - val_accuracy: 0.9973 -
val_loss: 0.1367 - learning_rate: 5.0000e-05
Epoch 43/100
563/563 ===== 96s 171ms/step - accuracy: 0.9975 - loss: 0.1047 - val_accuracy: 0.9973 -
val_loss: 0.1320 - learning_rate: 5.0000e-05
Epoch 44/100
563/563 ===== 96s 171ms/step - accuracy: 0.9975 - loss: 0.1072 - val_accuracy: 0.9973 -
val_loss: 0.1329 - learning_rate: 5.0000e-05
Epoch 45/100
563/563 ===== 0s 164ms/step - accuracy: 0.9976 - loss: 0.1090Saved weights to
model_weights_epoch_045.weights.h5
563/563 ===== 96s 171ms/step - accuracy: 0.9976 - loss: 0.1090 - val_accuracy: 0.9974 -
val_loss: 0.1311 - learning_rate: 5.0000e-05
Epoch 46/100
563/563 ===== 96s 170ms/step - accuracy: 0.9976 - loss: 0.1036 - val_accuracy: 0.9973 -
val_loss: 0.1321 - learning_rate: 2.5000e-05
Epoch 47/100
563/563 ===== 96s 170ms/step - accuracy: 0.9975 - loss: 0.1059 - val_accuracy: 0.9974 -
val_loss: 0.1323 - learning_rate: 2.5000e-05
Epoch 48/100
563/563 ===== 96s 170ms/step - accuracy: 0.9975 - loss: 0.1043 - val_accuracy: 0.9974 -
val_loss: 0.1335 - learning_rate: 2.5000e-05
Epoch 49/100
563/563 ===== 96s 170ms/step - accuracy: 0.9976 - loss: 0.1047 - val_accuracy: 0.9974 -
val_loss: 0.1326 - learning_rate: 2.5000e-05
Epoch 50/100
563/563 ===== 0s 163ms/step - accuracy: 0.9976 - loss: 0.1056Saved weights to
model_weights_epoch_050.weights.h5
563/563 ===== 96s 171ms/step - accuracy: 0.9976 - loss: 0.1056 - val_accuracy: 0.9973 -
val_loss: 0.1327 - learning_rate: 2.5000e-05

```

Resuming

Training ran across several Kaggle sessions, reloading the last periodic checkpoint and continuing with `initial_epoch`. The original notebook has one load/fit pair per session; the intermediate ones are omitted here and the final stretch is shown below.

The plateau is visible and worth stating plainly. Validation loss sits at ~0.130-0.132 from epoch 51 through 100 while the learning rate decays to 7.8e-07. The last 50 epochs bought essentially nothing. Early stopping should have fired; it did not because each resume restarted the patience counter. Training in one uninterrupted run, or persisting callback state across resumes, would have saved about half the compute.

```

history = model.fit(
    train_dataset,                # or train_gen if using generator
    validation_data=val_dataset,  # or val_gen 999999 \
    epochs=100,                  # total number of epochs you want to reach
    initial_epoch=90,            # resumes at epoch 41 if last saved was 40
    callbacks=callbacks,
    verbose=1
)

```

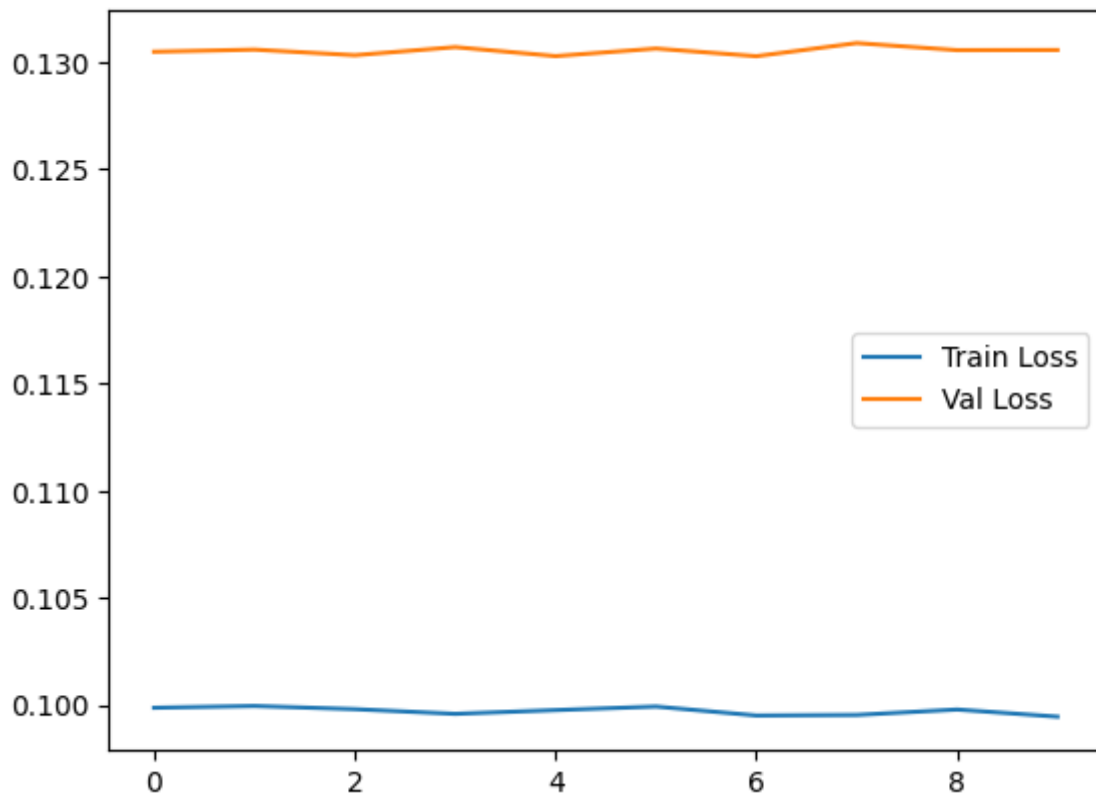
```

Epoch 91/100
563/563 ===== 25s 45ms/step - accuracy: 0.9976 - loss: 0.1001 - val_accuracy: 0.9974 - val_loss:
0.1305 - learning_rate: 1.5625e-06
Epoch 92/100
563/563 ===== 25s 45ms/step - accuracy: 0.9977 - loss: 0.0987 - val_accuracy: 0.9974 - val_loss:
0.1306 - learning_rate: 1.5625e-06
Epoch 93/100
563/563 ===== 25s 44ms/step - accuracy: 0.9977 - loss: 0.0999 - val_accuracy: 0.9974 - val_loss:
0.1303 - learning_rate: 1.5625e-06
Epoch 94/100
563/563 ===== 25s 44ms/step - accuracy: 0.9977 - loss: 0.0987 - val_accuracy: 0.9974 - val_loss:
0.1307 - learning_rate: 1.5625e-06
Epoch 95/100
563/563 ===== 0s 42ms/step - accuracy: 0.9975 - loss: 0.1002Saved weights to
model_weights_epoch_095.weights.h5
563/563 ===== 25s 45ms/step - accuracy: 0.9975 - loss: 0.1002 - val_accuracy: 0.9974 - val_loss:
0.1303 - learning_rate: 1.5625e-06
Epoch 96/100
563/563 ===== 25s 44ms/step - accuracy: 0.9976 - loss: 0.0998 - val_accuracy: 0.9974 - val_loss:
0.1306 - learning_rate: 1.5625e-06
Epoch 97/100
563/563 ===== 25s 44ms/step - accuracy: 0.9977 - loss: 0.0990 - val_accuracy: 0.9974 - val_loss:
0.1303 - learning_rate: 1.5625e-06
Epoch 98/100
563/563 ===== 25s 44ms/step - accuracy: 0.9977 - loss: 0.0985 - val_accuracy: 0.9974 - val_loss:
0.1309 - learning_rate: 1.5625e-06
Epoch 99/100
563/563 ===== 25s 44ms/step - accuracy: 0.9977 - loss: 0.1006 - val_accuracy: 0.9974 - val_loss:
0.1306 - learning_rate: 7.8125e-07
Epoch 100/100
563/563 ===== 0s 42ms/step - accuracy: 0.9976 - loss: 0.1002Saved weights to
model_weights_epoch_100.weights.h5
563/563 ===== 25s 45ms/step - accuracy: 0.9976 - loss: 0.1002 - val_accuracy: 0.9974 - val_loss:
0.1306 - learning_rate: 7.8125e-07

```

Loss curves

```
import matplotlib.pyplot as plt
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.legend()
plt.show()
```



7. Qualitative check

Three slices through one validation case: the four input modalities, the ground truth mask, and the prediction.

For a segmentation model this is the check that matters most. A single scalar metric cannot tell you *how* a model is wrong — whether it is missing the tumour core, over-segmenting the boundary, or producing scattered false positives. The contour comparison can.

```

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

# Make sure your model architecture and loss are defined
# model = build_3d_unet(input_shape=(48,48,48,4), n_filters=32, n_classes=3, dropout=0.3)
# model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4), loss=dice_loss,
metrics=['accuracy'])

# Load the best checkpoint
model.load_weights('best_model.h5')

# Get a batch from your validation dataset
X_sample, y_true_one_hot = next(iter(val_dataset)) # val_dataset as created before

# Predict segmentation on the first sample in the batch
y_pred_one_hot = model.predict(X_sample[0:1])

# Convert one-hot to class label (e.g., 0, 1, 2) for visualization
y_pred = np.argmax(y_pred_one_hot, axis=-1)[0] # shape: (48,48,48)
y_true = np.argmax(y_true_one_hot[0].numpy(), axis=-1) if hasattr(y_true_one_hot[0], 'numpy') else
np.argmax(y_true_one_hot[0], axis=-1)

# Choose some slices (25%, 50%, 75% along z-axis)
slice_indices = [12, 24, 36]
modalities = ['t1n', 't1c', 't2w', 't2f']

fig, axs = plt.subplots(len(modalities)+2, len(slice_indices), figsize=(12, 16))

# Show modalities
for i, modality in enumerate(modalities):
    for j, slc in enumerate(slice_indices):
        axs[i, j].imshow(X_sample[0, :, :, slc, i], cmap='gray')
        axs[i, j].axis('off')
        if j == 0:
            axs[i, j].set_ylabel(modality, fontsize=12)

# Show ground truth mask
for j, slc in enumerate(slice_indices):
    axs[len(modalities), j].imshow(y_true[:, :, slc], cmap='viridis')
    axs[len(modalities), j].axis('off')
    if j == 0:
        axs[len(modalities), j].set_ylabel('True Mask', fontsize=12)

# Show predicted mask
for j, slc in enumerate(slice_indices):
    axs[len(modalities)+1, j].imshow(y_pred[:, :, slc], cmap='viridis')
    axs[len(modalities)+1, j].axis('off')
    if j == 0:
        axs[len(modalities)+1, j].set_ylabel('Pred Mask', fontsize=12)

plt.tight_layout()
plt.show()

```

```

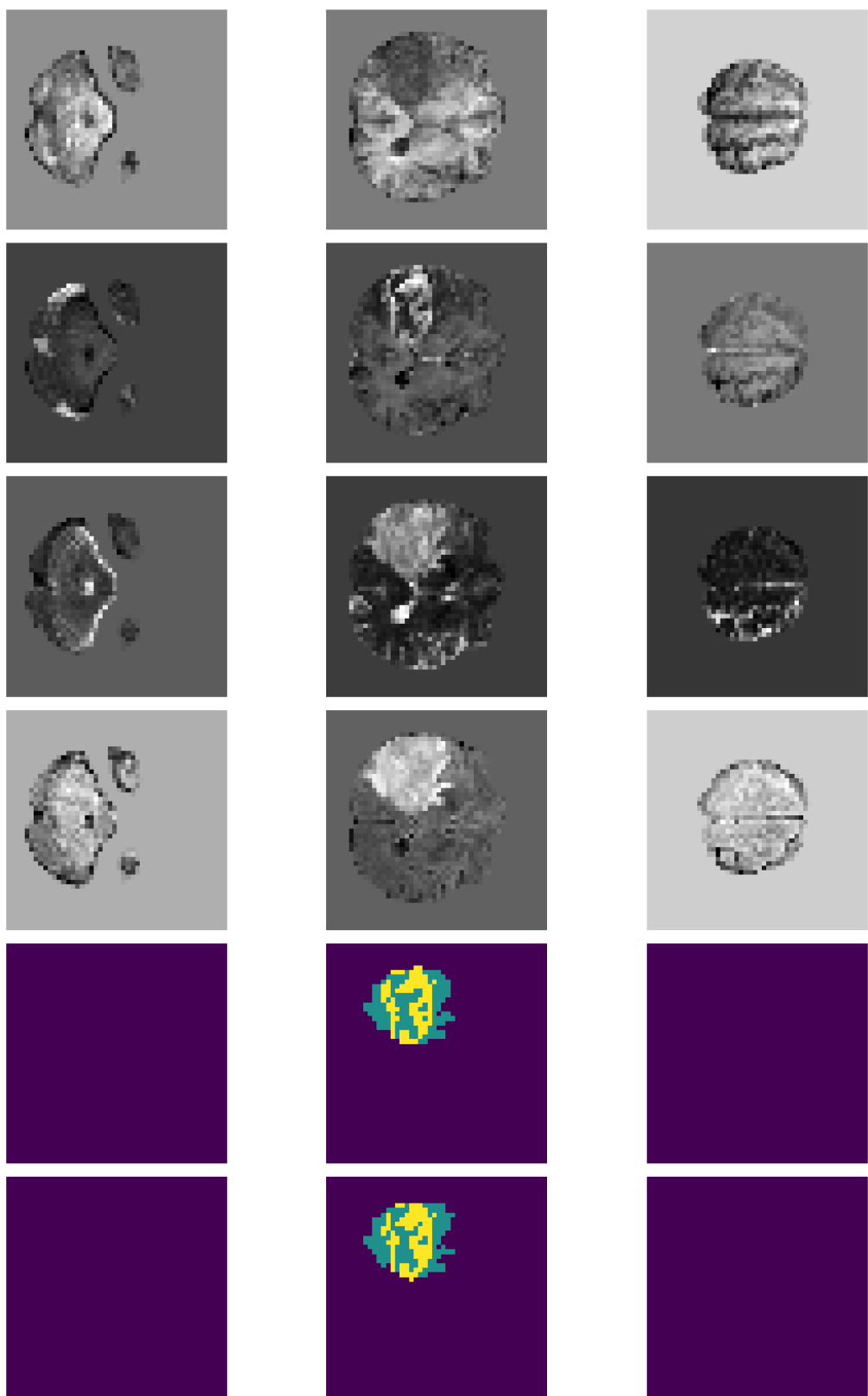
/usr/local/lib/python3.11/dist-packages/keras/src/models/functional.py:237: UserWarning: The structure of
`inputs` doesn't match the expected structure.
Expected: ['keras_tensor']
Received: inputs=Tensor(shape=(1, 48, 48, 48, 4))
warnings.warn(msg)

```

```

1/1 ===== 2s 2s/step

```

8. Evaluation as originally run — and why the numbers are wrong

This is the section to read carefully.

The evaluation below merges every tumour class into one binary mask ($\text{mask} > 0$) and then computes Dice **per 2D slice**, averaging over all slices of all validation volumes.

The output of the next cell shows the problem in miniature.

```
import numpy as np

def dice_score(y_true, y_pred, smooth=1e-6):
    y_true_f = y_true.flatten()
    y_pred_f = y_pred.flatten()
    intersection = np.sum(y_true_f * y_pred_f)
    return (2. * intersection + smooth) / (np.sum(y_true_f) + np.sum(y_pred_f) + smooth)

def iou_score(y_true, y_pred, smooth=1e-6):
    y_true_f = y_true.flatten()
    y_pred_f = y_pred.flatten()
    intersection = np.sum(y_true_f * y_pred_f)
    union = np.sum(y_true_f) + np.sum(y_pred_f) - intersection
    return (intersection + smooth) / (union + smooth)

# To calculate for ALL tumor regions combined (rather than individual classes)
def binary_mask(mask):
    return (mask > 0).astype(np.uint8)

slice_indices = [12, 24, 36] # Example: same slices you visualized
dice_scores = []
iou_scores = []

for slc in slice_indices:
    true_mask_slice = binary_mask(y_true[:, :, slc]) # Use your arrays here!
    pred_mask_slice = binary_mask(y_pred[:, :, slc])
    dice_scores.append(dice_score(true_mask_slice, pred_mask_slice))
    iou_scores.append(iou_score(true_mask_slice, pred_mask_slice))

print("Dice scores:", dice_scores)
print("IoU scores:", iou_scores)
```

```
Dice scores: [1.0, 0.9808102345824942, 1.0]
IoU scores: [1.0, 0.9623430963918699, 1.0]
```

[1.0, 0.9808, 1.0]

Two of those three slices scored exactly 1.0. They contain no tumour — in either the ground truth or the prediction. For an empty slice the Dice expression becomes

$$(2 * 0 + 1e-6) / (0 + 0 + 1e-6) = 1.0$$

The smoothing term, which exists to avoid dividing by zero, hands out a perfect score for correctly predicting nothing. That is not a segmentation result.

Most axial slices of a brain contain no tumour. So when the same metric is run over the full validation set, the mean is dominated by free 1.0s.

```

import numpy as np

def dice_score(y_true, y_pred, smooth=1e-6):
    y_true_f = y_true.flatten()
    y_pred_f = y_pred.flatten()
    intersection = np.sum(y_true_f * y_pred_f)
    return (2. * intersection + smooth) / (np.sum(y_true_f) + np.sum(y_pred_f) + smooth)

def iou_score(y_true, y_pred, smooth=1e-6):
    y_true_f = y_true.flatten()
    y_pred_f = y_pred.flatten()
    intersection = np.sum(y_true_f * y_pred_f)
    union = np.sum(y_true_f) + np.sum(y_pred_f) - intersection
    return (intersection + smooth) / (union + smooth)

def binary_mask(mask):
    # Combine all tumor regions into one binary mask for global Dice/IoU.
    return (mask > 0).astype(np.uint8)

all_dice_scores = []
all_iou_scores = []

# Loop through the entire validation set
for X_batch, y_batch_true in val_dataset:
    y_batch_pred = model.predict(X_batch) # Your trained model's predictions
    batch_size = X_batch.shape[0]
    for i in range(batch_size):
        y_true_labels = np.argmax(y_batch_true.numpy()[i], axis=-1) # shape: (48, 48, 48)
        y_pred_labels = np.argmax(y_batch_pred[i], axis=-1)
        for slc in range(y_true_labels.shape[2]):
            y_true_slice = binary_mask(y_true_labels[:, :, slc])
            y_pred_slice = binary_mask(y_pred_labels[:, :, slc])
            dice = dice_score(y_true_slice, y_pred_slice)
            iou = iou_score(y_true_slice, y_pred_slice)
            all_dice_scores.append(dice)
            all_iou_scores.append(iou)

mean_dice = np.mean(all_dice_scores)
mean_iou = np.mean(all_iou_scores)

print(f"Mean Dice: {mean_dice}")
print(f"Mean IoU: {mean_iou}")

```

```

1/1 ===== 2s 2s/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 47ms/step
1/1 ===== 0s 55ms/step
1/1 ===== 0s 55ms/step
1/1 ===== 0s 55ms/step
1/1 ===== 0s 52ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 54ms/step
1/1 ===== 0s 49ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 47ms/step
1/1 ===== 0s 50ms/step
1/1 ===== 0s 50ms/step
1/1 ===== 0s 49ms/step
1/1 ===== 0s 49ms/step
1/1 ===== 0s 51ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 52ms/step
1/1 ===== 0s 51ms/step
1/1 ===== 0s 50ms/step
1/1 ===== 0s 49ms/step
1/1 ===== 0s 47ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 49ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 47ms/step
1/1 ===== 0s 48ms/step

```

```

1/1 ===== 0s 53ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 50ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 49ms/step
1/1 ===== 0s 51ms/step
1/1 ===== 0s 49ms/step
1/1 ===== 0s 47ms/step
1/1 ===== 0s 47ms/step
1/1 ===== 0s 47ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 49ms/step
1/1 ===== 0s 50ms/step
1/1 ===== 0s 50ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 50ms/step
1/1 ===== 0s 52ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 49ms/step
1/1 ===== 0s 50ms/step
1/1 ===== 0s 49ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 48ms/step
1/1 ===== 0s 48ms/step
Mean Dice: 0.9358623704163483
Mean IoU: 0.9112667523132757

```

The reported 0.936 Dice / 0.911 IoU

That is where the numbers on my CV came from, and there are three independent reasons not to trust them:

1. **Empty slices score 1.0.** Every tumour-free slice contributes a perfect score via the smoothing term. This is the dominant effect.
2. **It is whole-tumour, not sub-region.** `binary_mask` collapses all classes into one blob. Whole-tumour Dice is always substantially higher than per-class Dice, and enhancing tumour is the hard class.
3. **It is 2D per-slice, on 48³ volumes.** BraTS reports volumetric Dice at full resolution. Neither the dimensionality nor the resolution matches.

Add the label-merging bug from Section 2 and the number describes something other than what it claims to.

9. Corrected evaluation

Four changes:

- **Volumetric, not per-slice.** Dice is computed over the whole 3D mask.
- **Per class, not merged.** Each class is scored separately, so a weak class cannot hide behind a strong one.
- **Absent-in-both returns NaN and is excluded**, rather than scoring 1.0. The count of scored cases is reported alongside each mean so the reader can see how much data each number rests on.
- **No smoothing constant in the reported metric.** Smoothing belongs in the loss, where it stabilises gradients. In a reported metric it manufactures scores.

These cells have not been re-executed — the trained weights are not in this session. They are the corrected code, not corrected results. The honest statement of this project's outcome is therefore: a 3D U-Net trained to convergence on downsampled BraTS 2025 volumes, with the quantitative evaluation still to be redone properly.

```
import numpy as np

def dice_and_iou(y_true_binary, y_pred_binary):
    """Volumetric Dice and IoU for one binary mask pair.

    Returns (nan, nan) when the class is absent from BOTH ground truth and
    prediction. Scoring an empty-empty case as 1.0 is what inflated the
    original numbers.
    """
    true_sum = y_true_binary.sum()
    pred_sum = y_pred_binary.sum()

    if true_sum == 0 and pred_sum == 0:
        return np.nan, np.nan

    intersection = np.logical_and(y_true_binary, y_pred_binary).sum()
    union = true_sum + pred_sum - intersection

    dice = 2.0 * intersection / (true_sum + pred_sum)
    iou = intersection / union if union > 0 else np.nan
    return dice, iou
```

```

CLASS_NAMES = ("background", "class_1", "class_2")

def evaluate(model, dataset, n_classes=3, class_names=CLASS_NAMES):
    """Per-class volumetric Dice and IoU across a dataset."""
    scores = {c: {"dice": [], "iou": []} for c in range(n_classes)}

    for x_batch, y_batch in dataset:
        y_pred_batch = model.predict(x_batch, verbose=0)

        for i in range(x_batch.shape[0]):
            y_true = np.argmax(y_batch[i].numpy(), axis=-1)
            y_pred = np.argmax(y_pred_batch[i], axis=-1)

            for c in range(n_classes):
                dice, iou = dice_and_iou(y_true == c, y_pred == c)
                if not np.isnan(dice):
                    scores[c]["dice"].append(dice)
                    scores[c]["iou"].append(iou)

    print(f"{'class':<16}{'n cases':>9}{'Dice':>9}{'IoU':>9}")
    print("-" * 43)

    results = {}
    for c in range(n_classes):
        dices = scores[c]["dice"]
        ious = [v for v in scores[c]["iou"] if not np.isnan(v)]
        if not dices:
            print(f"{class_names[c]:<16}{0:>9}{'absent':>9}{'absent':>9}")
            continue
        results[class_names[c]] = {
            "dice": float(np.mean(dices)),
            "iou": float(np.mean(ious)),
            "n": len(dices),
        }
        print(f"{class_names[c]:<16}{len(dices):>9}"
              f"{np.mean(dices):>9.4f}{np.mean(ious):>9.4f}")

    foreground = [v["dice"] for k, v in results.items() if k != "background"]
    if foreground:
        print("-" * 43)
        print(f"{'mean foreground':<16}{'':>9}{np.mean(foreground):>9.4f}")

    return results

# model.load_weights('best_model.h5')
# results = evaluate(model, val_dataset)

```

10. What I would change

Ordered by how much each would actually matter.

1. Fix the label remap. Two tissue classes are currently merged. Train on all four BraTS labels, or merge deliberately into the standard whole-tumour / tumour-core / enhancing-tumour regions that the challenge actually scores.

2. Re-run the corrected evaluation and report per-class volumetric Dice. Enhancing tumour separately, because it is the class that discriminates between methods.

3. Train at higher resolution using patches rather than downsampling whole volumes to 48^3 . Patch-based training keeps native resolution at the cost of losing global context, which is the standard trade in this problem.

4. Weight the loss by class, or use Dice plus focal loss. Small classes are still under-served even with Dice.

5. Persist callback state across session resumes. Would have cut training compute roughly in half, per the plateau in Section 6.

6. Replace the `tf.py_function` loader with TFRecords. The current pipeline is GIL-bound.

7. Report uncertainty. A single Dice number per class says nothing about variance across cases. Per-case distributions, or at minimum an interquartile range, would be more informative — and for anything decision-critical, a calibrated confidence estimate matters more than the mean.